



서브쿼리 내에서 대표적으로 조건 비교하는 where, having -> 집계함수 응용할 때
> , < , In ,Any ALL 많이 사용

집계 함수



06-3 집계 함수

■ 조건에 맞는 데이터 개수를 세는 함수 — COUNT

1. 조건에 맞는 데이터의 개수를 세고 싶다면 COUNT 함수를 사용함.
이때, COUNT 함수가 반환하는 데이터의 범위는 BIGINT임.

다음은 customer 테이블의 행 개수를 모두 집계한 쿼리임.
결과를 살펴보면 행(데이터) 개수가 총 599개라는 것을 알 수 있음

Do it! COUNT 함수로 데이터 개수 집계

```
SELECT COUNT(*) FROM customer;
```

실행 결과

	COUNT(*)
▶	599



06-3 집계 함수

■ 조건에 맞는 데이터 개수를 세는 함수 — COUNT

- 이번에는 store_id 열 기준으로 그룹화하여 store_id 열의 그룹마다 데이터가 몇 개씩 있는지 확인하는 쿼리를 작성해 보자.

Do it! COUNT 함수와 GROUP BY 문 조합

```
SELECT store_id, COUNT(*) AS cnt FROM customer GROUP BY  
store_id;
```

실행 결과

	store_id	cnt
▶	1	326
	2	273

이와 같이 customer 테이블의 store_id 열에 있는 데이터를
GROUP BY 문으로 묶은 뒤,
각 그룹별 데이터 개수를 출력한 것을 확인할 수 있음



06-3 집계 함수

■ 조건에 맞는 데이터 개수를 세는 함수 — COUNT

3. 계속해서 store_id 열과 active 열 기준으로 그룹화해서 store_id 열과 active 열 그룹마다 데이터가 각각 몇 개씩 있는지 확인하는 쿼리를 작성해 보자

Do it! COUNT 함수와 GROUP BY 문 조합: 열 2개 활용

```
SELECT store_id, active, COUNT(*) AS cnt FROM customer GROUP BY store_id, active;
```

실행 결과

	store_id	active	cnt
▶	1	1	318
	2	1	266
	2	0	7
	1	0	8



06-3 집계 함수

■ 조건에 맞는 데이터 개수를 세는 함수 — COUNT

4. COUNT 함수를 사용할 때 주의할 점이 있음.

COUNT 함수에 전체 열이 아닌 특정 열만 지정하여 집계할 때
해당 열에 있는 NULL값은 제외함.

그래서 전체 데이터 개수와 COUNT 함수로 얻은 데이터 개수가 다를 수 있음



06-3 집계 함수

■ 조건에 맞는 데이터 개수를 세는 함수 — COUNT

4. 다음은 address 테이블 전체의 행 개수와 address2 열의 행 개수를 센 것임.

결과를 보면 address2 열의 NULL은
COUNT 함수의 집계 대상이 아니므로 전체 행 개수와 다름

Do it! NULL을 제외한 집계 확인

```
SELECT COUNT(*) AS all_cnt,  
COUNT(address2) AS ex_null FROM address;
```

실행 결과

	all_cnt	ex_null
▶	603	599



06-3 집계 함수

■ 조건에 맞는 데이터 개수를 세는 함수 — COUNT

5. COUNT 함수를 사용할 때 DISTINCT 문을 조합하면 중복된 값을 제외한 데이터 개수를 집계할 수 있음.

다음 쿼리는 customer 테이블에서
전체 행 개수와 store_id 열의 행 개수
그리고 store_id 열에서 중복된 값을 제외한 행 개수를 조회함



06-3 집계 함수

■ 데이터의 합을 구하는 함수 — SUM

1. 다음은 payment 테이블에 있는 amount 열의 모든 데이터를 더하는 쿼리임.

Do it! SUM 함수로 amount 열의 데이터 합산

```
SELECT SUM(amount) FROM payment;
```

실행 결과

SUM(amount)
67406.56

이와 같이 SUM 함수의 인자로 열 이름을 넣으면
그 열에 있는 숫자 데이터를 모두 합산한 결과를 얻을 수 있음



06-3 집계 함수

■ 데이터의 합을 구하는 함수 — SUM

2. 계속해서 다른 쿼리도 입력해 보자.
다음은 SUM 함수와 GROUP BY 문을 조합하여 amount 열의 데이터를 합산하되 customer_id 열의 데이터를 그룹으로 나누어 합산하는 쿼리임.

Do it! SUM 함수와 GROUP BY 문 조합

```
SELECT customer_id, SUM(amount) FROM payment GROUP BY customer_id;
```

실행 결과를 살펴보면
전체 16044행에서 599개 행이
customer_id로 그룹화되고,
각 그룹에 해당하는 amount 열의 값이
더해진 결과를 얻을 수 있음

실행 결과

	customer_id	SUM(amount)
▶	1	118.68
	2	128.73
	3	135.74
	4	81.78
	5	144.62
	6	93.72
	7	151.67
	8	92.76
	9	89.77
	10	99.75



06-3 집계 함수

■ 데이터의 합을 구하는 함수 — SUM

3. 만약 SUM 함수를 사용했을 때,
SUM 함수로 합산한 데이터가 기존 데이터 유형의 범위를 넘는 경우가 있음.

보통은 암시적 형 변환으로 오류가 발생하지는 않았지만
형 변환된 데이터 유형으로 결과가 반환된 것을 볼 수 있음.

다음 쿼리는 정수(int)로 된 데이터들을 더함.
col_1열의 데이터를 합산한 결과가 30억을 넘어 오버플로가 발생해야 정상임.

하지만 MySQL에서는 암시적 형 변환으로
DECIMAL 또는 BIGINT로 합산한 결과를 반환함



06-3 집계 함수

■ 데이터의 합을 구하는 함수 — SUM

3.

Do it! 암시적 형 변환으로 오버플로 없이 합산 결과를 반환

```
CREATE TABLE doit_overflow (  
col_1 int,  
col_2 int,  
col_3 int  
);  
  
INSERT INTO doit_overflow VALUES (1000000000,1000000000, 1000000000);  
INSERT INTO doit_overflow VALUES (1000000000,1000000000, 1000000000);  
INSERT INTO doit_overflow VALUES (1000000000,1000000000, 1000000000);  
SELECT SUM(col_1) FROM doit_overflow;
```

실행 결과

SUM(col_1)
3000000000



06-3 집계 함수

■ 데이터의 평균을 구하는 함수 — AVG

1. 다음은 payment 테이블에서 amount 열의 데이터 평균을 구하는 쿼리임.

Do it! AVG 함수로 amount 열의 데이터 평균 계산

```
SELECT AVG(amount) FROM payment;
```

실행 결과

AVG(amount)
4.201356

이와 같이 AVG 함수의 인자로 열 이름을 넣으면
그 열에 있는 숫자 데이터의 평균을 계산한 결과를 얻을 수 있음



06-3 집계 함수

■ 데이터의 평균을 구하는 함수 — AVG

- 이번에는 AVG 함수와 GROUP BY 문을 조합해 customer_id 열을 그룹화하고, 각 그룹별로 amount 열의 데이터 평균을 계산하는 쿼리를 작성해 보자.

Do it! AVG 함수와 GROUP BY 문 조합

```
SELECT customer_id, AVG(amount) FROM payment GROUP BY customer_id;
```

결과를 살펴보면 SUM 함수에서 다뤘던 것과 동일하게 customer_id로 그룹화되고, 각 그룹의 평균값을 나타냄

실행 결과

	customer_id	AVG(amount)
▶	1	3.708750
	2	4.767778
	3	5.220769
	4	3.717273
	5	3.805789
	6	3.347143
	7	4.596061
	8	3.865000
	9	3.903043
	10	3.990000



06-3 집계 함수

■ 최솟값 또는 최댓값을 구하는 함수 — MIN, MAX

1. 이번에도 역시 payment 테이블에 있는 amount 열의 최솟값, 최댓값을 조회하는 쿼리임.

Do it! MIN과 MAX 함수로 amount 열의 최솟값과 최댓값 조회

```
SELECT MIN(amount), MAX(amount) FROM payment;
```

실행 결과

	MIN(amount)	MAX(amount)
▶	0.00	11.99

이와 같이 MIN 또는 MAX 함수의 인자로 열 이름을 넣으면
그 열에 있는 숫자 데이터 중 최솟값 또는 최댓값을 각각 조회한 것을 확인할 수 있음



06-3 집계 함수

■ 최솟값 또는 최댓값을 구하는 함수 — MIN, MAX

2. 다른 집계 함수처럼 그룹화된 데이터들 중에서 최솟값과 최댓값을 구할 수 있음.
다음은 customer_id의 그룹별로 최솟값, 최댓값을 구한 쿼리임.

Do it! MIN과 MAX 함수 그리고 GROUP BY 문 조합

```
SELECT customer_id, MIN(amount), MAX(amount) FROM  
payment GROUP BY customer_id;
```

결과를 살펴 보면 customer_id별로 그룹화하고,
각 customer_id에 따라 최솟값과 최댓값이
조회된 것을 확인할 수 있음

실행 결과

	customer_id	MIN(amount)
▶	1	0.99
	2	0.99
	3	0.99
	4	0.99
	5	0.99
	6	0.99
	7	0.99
	8	0.99
	9	0.99
	10	0.99



06-3 집계 함수

■ 부분합과 총합을 구하는 함수 — ROLLUP

- 데이터의 부분합과 총합을 구하고 싶다면
GROUP BY 문과 ROLLUP 함수에 조합하면 됨
- ROLLUP 함수는 **GROUP BY (열 이름) WITH ROLLUP**에서 입력한 열을 기준으로
오른쪽에서 왼쪽으로 이동하면서 부분합과 총합을 구함

Do it! ROLLUP 함수로 부분합 계산

```
SELECT customer_id, staff_id, SUM(amount)
FROM payment
GROUP BY customer_id, staff_id WITH
ROLLUP;
```

실행 결과

	customer_id	staff_id	SUM(amount)
▶	1	1	64.83
	1	2	53.85
	1	NULL	118.68
	2	1	60.85
	2	2	67.88
	2	NULL	128.73
	3	1	64.86
	3	2	70.88
	3	NULL	135.74
	4	1	49.88

596	1	54.83
596	2	41.89
596	NULL	96.72
597	1	49.86
597	2	49.89
597	NULL	99.75
598	1	43.90
598	2	39.88
598	NULL	83.78
599	1	28.92
599	2	54.89
599	NULL	83.81
NULL	NULL	67406.56



06-3 집계 함수

■ 부분합과 총합을 구하는 함수 — ROLLUP

- 결과를 보면 customer_id와 staff_id 열을 그룹화한 것의 부분합이라는 것을 알 수 있음
- 예를 들어 3행은 customer_id가 10이고, staff_id 열에 속한 데이터의 총합인 셈임
- 오른쪽에서 왼쪽으로 이동한다고 했던 것은 staff_id가 변하면서 customer_id가 변하지 않는 것을 보면 이해할 수 있음



06-3 집계 함수

■ 부분합과 총합을 구하는 함수 — ROLLUP

- 마지막 행을 확인해 보면 customer_id, staff_id 모두 NULL인 행의 전체 총합을 알 수 있음
- ROLLUP을 사용할 때 GROUP BY 뒤의 순서에 따라 계층화 부분도 달라짐
- 그러므로 계층화해서 합계를 보려면 열의 순서를 잘 정할 수 있어야 함



06-3 집계 함수

■ 데이터의 표준편차를 구하는 함수 — STDDEV, STDDEV_SAMP

- 표준편차를 구하려면 STDDEV 또는 STDDEV_SAMP 함수를 사용하면 됨
- STDDEV 함수는 모든 값에 대한 표준편차를,
STDDEV_SAMP 함수는 표본에 대한 표준편차를 구함.

Do it! STDDEV와 STDDEV_SAMP 함수로 표준편차 계산

```
SELECT STDDEV(amount), STDDEV_SAMP(amount) FROM  
payment;
```

실행 결과

	STDDEV(amount)	STDDEV_SAMP(amount)
▶	2.3628869202372846	2.3629605613923124

이와 같이 STDDEV와 STDDEV_SAMP 함수의 인자로 열 이름을 넣으면
그 열에 있는 숫자 데이터의 표준편차를 얻을 수 있음